



PROGRAMMING FUNDAMENTALS

CS – 101

Instructor: Maria N. Samad

November 2nd, 2022

November 11th, 2022

LIST OPERATIONS

- Concatenation: Using the + operator like Strings

- For example:

- A = [1, 2, 3]

- B = [4, 5, 6]

- C = A + B

- print(C)

→ Output = ?

- Repetition: Using the * operator like Strings

- For example:

- List1 = [0]

- print (List1 * 4)

→ Output?

- A = [1, 2, 3]

- print(A * 3)

→ Output?

LIST SLICES

- The slice operator works on Lists similar to Strings
- For example:
 - `T = ['a', 'b', 'c', 'd', 'e', 'f']`
 - `print(T[1:3])` → Output ?
 - `print(T[:4])` → Output ?
 - `print(T[3:])` → Output ?
 - `print(T[ : ])` → Output ?

LIST SLICES

- Unlike strings, you can modify elements here
- For example:
 - `T = ['a', 'b', 'c', 'd', 'e', 'f']`
`T[1:3] = ['x', 'y']`
`print(T)`
- Using step function in lists, just like strings
- For example:
 - `l1 = [0,1,2,3,4,5,6,7,8,9,10,11,12,13]`
`print(l1[2:5])`
`print(l1[0:10:2])`
`print(l1[5:0:-2])`

LIST METHODS

- ***append***

- Adds a new element to the end of a list

- For example,

- `T = ['a', 'b', 'c']`
`T.append('d')`
`print(T)`

→ Output?

LIST METHODS

- ***extend***

- What if we want to add more than one element?
- This method takes a ***list*** as an argument, and appends those elements in the given list
- The argument list remains unchanged
- For example,
 - T1 = ['a', 'b', 'c']
T2 = ['d', 'e']
T1.extend(T2)
print(T1) → Output?
print(T2) → Output?

LIST METHODS

- ***sort***

- Arranges the elements of the list from low to high

- For example,

- `T = ['d', 'c', 'e', 'b', 'a']`

- `T.sort()`

- `print(T)`

→ Output?

- All the list methods are void functions, i.e. they do not return any value

- `T = T.sort()`

→ Output?

LIST METHODS

- ***pop***

- Removing an element w.r.t its index
- It modifies the list and returns the element that was removed
- If no index specified, it assumes the last index
- For example,

- `T = ['a', 'b', 'c', 'd', 'e']`

- `x = T.pop(1)`

- `print(T)`

→ Output?

- `print(x)`

→ Output?

- `y = T.pop()`

- `print(T)`

→ Output?

- `print(y)`

→ Output?

LIST METHODS

- *del*

- Removing an element w.r.t its index
- It modifies the list but does NOT return anything
- Index is ALWAYS required
- For example,

- `T = ['a', 'b', 'c', 'd', 'e']`

- `del(T[1])`

- `print(T)`

→ Output?

- `del(T[])`

- `print(T)`

→ Output?

LIST METHODS

- ***remove***

- Removing an element w.r.t the element value, and not the index
- In case of multiple elements, it removes the first access of the element value from the list
- It modifies the list but does NOT return anything

- For example,

- `T = ['a', 'b', 'c', 'd', 'e']`

- `T.remove('b')`

- `print(T)`

→ Output?

- `x = T.remove('c')`

- `print(T)`

→ Output?

LIST METHODS

- Deleting more than one elements in the list
 - Can only be applied to indices
 - Use slice operator with ***del***
 - For example,
 - `T = ['a', 'b', 'c', 'd', 'e']`
`del(T[1:5])`
`print(T)` → Output?

LIST AND STRINGS

- A string is a sequence of characters
- A list is a sequence of values (which may or may not be characters)
- To convert from a string to a list of characters, use the ***list*** method:
 - `s = "spam"`
`t = list(s)`
`print(t)` → Output?
- ***list*** is a built-in function so don't use it as variable name

LIST AND STRINGS

- ***split***

- Breaks strings into words
- String is separated by blank spaces
- For example,
 - `s = "Pining for the fjords"`
`t = s.split()`
`print(t)` → Output?
- Can use other characters as a separator and still break the string into words using the ***delimiter*** keyword
 - `s = "spam-spam-spam"`
`delimiter = '-'`
`t = s.split(delimiter)`
`print(t)` → Output?

LIST AND STRINGS

- *join*

- The opposite functionality of split
- Concatenates the words into a string
- Explicitly needs to mention delimiter
- For example,
 - `T = ["pining", "for", "the", "fjords"]`
`delimiter = ' '`
`s = delimiter.join(T)`
`print(s)` → Output?

OBJECTS AND VALUES

- *is* operator:
 - To check whether two string variables refer to the same object, you can use the *is* operator
 - For example,
 - `a = "banana"`
`b = "banana"`
`print(a is b)` → Output?
`print(b is a)` → Output?
 - Here Python creates one string object, and both **a** and **b** refer to it

OBJECTS AND VALUES

- *is* operator:
 - Not true for lists though
 - For example,
 - `a = [1, 2, 3, 4]`
`b = [1, 2, 3, 4]`
`print(a is b)` → Output?
`print(b is a)` → Output?
- The lists are consider **equivalent** because they have the same elements, but not **identical** as they are not referring to the same object
- Association by **value**

OBJECTS AND VALUES

- *is* operator:
 - However, when a variable is assigned the same value through another variable
 - For example,
 - `a = "banana"`
`b = a`
`print(a is b)` → Output?
 - `a = [1, 2, 3, 4]`
`b = a`
`print(a is b)` → Output?
- This is called association by **reference**

OBJECTS AND VALUES

- *is* operator:
 - Changes in one object, reflects in the associated object as well, ONLY if the objects are mutable
 - For example,
 - `a = [1, 2, 3, 4]`
`b = a`
`b[0] = 17`
`print(b)` → Output?
`print(a)` → Output?
 - For immutable objects like strings, this is not applicable

LIST ARGUMENTS

- Passing lists as arguments to functions
- The function gets a **reference** to the list
- Any changes made to the list inside the function will be visible to the caller function as well
- For example,
 - ```
def delete_head(t):
 del t[0]
```

```
letters = ['a', 'b', 'c', 'd']
delete_head(letters)
print(letters)
```

→ Output?

# LIST ARGUMENTS

- Example:
  - ```
def passbyref(x, l1):  
    x = x + 1  
    l1[0] = 67  
    print("Inside function, single variable:",x)  
    print("Inside function, list:",l1)  
  
def main():  
    lst1 = [10, 20, 30, 40]  
    num1 = 3.5  
    print("Single variable before calling function:",num1)  
    print("List before calling function:",lst1)  
  
    passbyref(num1,lst1)  
  
    print("Single variable after calling function:",num1)  
    print("List after calling function:",lst1)  
  
if __name__ == "__main__":  
    main()
```

NESTED LIST

- A list within another list
- A kind of sub-list that will appear as an element to the outer list
- Examples:
 - `nested = ["hello", 3.5, 5, [12, 23]]`
 - `print(nested[3])` -- > output = ?
 - `elem = nested[3]`
 - `print(elem)` -- > output = ?
 - `print(elem[0])` -- > output = ?
 - `print(elem[1])` -- > output = ?
 - `print(nested[3][0])` -- > output = ?

NESTED LIST

- Matrices are often represented by using nested lists

- For a given matrix $\begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{bmatrix}$

- We would have a nested list to represent it in Python,
i.e. `[ [1, 2, 3], [4, 5, 6], [7, 8, 9] ]`

NESTED LIST

- Example: Define a square matrix of size n given as input to the function, and initialize it with all zeros
- Solution 1:
 - ```
def matrix_correct(n):
 m = []
 for j in range(n):
 lst = []
 for i in range(n):
 lst.append(0)
 m.append(lst)
 return m
```

# NESTED LIST

- Example: Define a square matrix of size ***n*** given as input to the function, and initialize it with all zeros
- Solution 2:
  - ```
def matrix_incorrect(n):  
    lst = []  
    for i in range(n):  
        lst.append(0)  
    m = []  
    for j in range(n):  
        m.append(lst)  
    return m
```


NESTED LIST

- Example: Define a square matrix of size n given as input to the function
- At first the two solutions may yield the same results, but one method is correct and the other incorrect
 - Association by reference!

NESTED LIST

- Example: Define a square matrix of size n given as input to the function
- Try updating the matrices:
- Main program:
 - `mi = matrix_incorrect(3)`
`mi[0][0] = 1`
`print(mi)` → Output?
 - `mc = matrix_correct(3)`
`mc[0][0] = 1`
`print(mc)` → Output?

EXAMPLES

- Write a function called `matrix_sum` that takes a 4x4 matrix as argument and adds up all the elements of the matrix, and return the sum in the end
- Solution:
 - ```
def matrix_sum(m):
 total = 0
 for i in range (len(m)):
 n = m[i]
 for j in range(len(n)):
 total = total + n[j]
 return total
```
  - Main program:
    - ```
m = [[1,2,3,4], [5,6,7,8], [9,10,11,12],[13,14,15,16]]  
t = matrix_sum(m)  
print(t)
```

 → Output?

EXAMPLES

- Write a function called `capitalize_nl` that takes a nested list of lower case letters as argument and returns a new nested list with all the letters in upper case {The returned list is of the same size and order}

- Solution:

```
def capitalize_nl(nl):  
    new_nl = []  
    for i in range(len(nl)):  
        temp_nl = []  
        elem = nl[i]  
        for j in range (len(elem)):  
            c = elem[j].capitalize()  
            temp_nl.append(c)  
        new_nl.append(temp_nl)  
    return new_nl
```

EXAMPLES

- Write a function called `capitalize_nl` that takes a nested list of lower case letters as argument and returns a new nested list with all the letters in upper case {The returned list is of the same size and order}
- Solution:
 - Main program:
 - ```
nl = [['a', 'b', 'c'], ['d', 'e'], 'f', ['g', 'h', 'i', 'j']]
nnl = capitalize_nl(nl)
print(nnl)
```

 → Output?

# SELF PRACTICE TASKS

- Modify the last function to add condition for `isupper()`. If true, then no need to capitalize, just directly insert in the new list, otherwise capitalize and then insert
- Write a function called ***cumulative\_sum*** that takes a nested list of numbers and returns a new nested list where it's i-th element is the sum of i – 1 elements from the argument list
  - For example, if input list is:  
    `[[1,2,3], [4,5,6,0], [7,8,9,1]]`  
then output will be:  
    `[[1,3,6], [4,9,15,15], [7,15,24,25]]`

# SELF PRACTICE TASKS

- Write a function called ***middle\_m*** that takes a nested list as argument, and return a new nested list that contains all elements **BUT** first and last elements for each row
  - For example, if input is:  
[[1,2,3], [4,5,6,7], [8,9], [10,11,11,10]]  
Then the output will be:  
[[2], [5,6], [ ], [11,11]]